

RFist - 네트워크 입력 동기화 코드 샘플

로컬 입력 캐시에서 네트워크 입력 수집, 캐릭터 액션 실행까지

멀티플레이 액션 게임에서 입력 데이터 정의, 로컬 입력 수집, 네트워크 전송, 수신 입력 처리, 실제 캐릭터 동작 실행까지 이어지는 흐름을 보여주는 코드 샘플입니다.

Multiplayer

Input Authority

Network Input

Character Controller

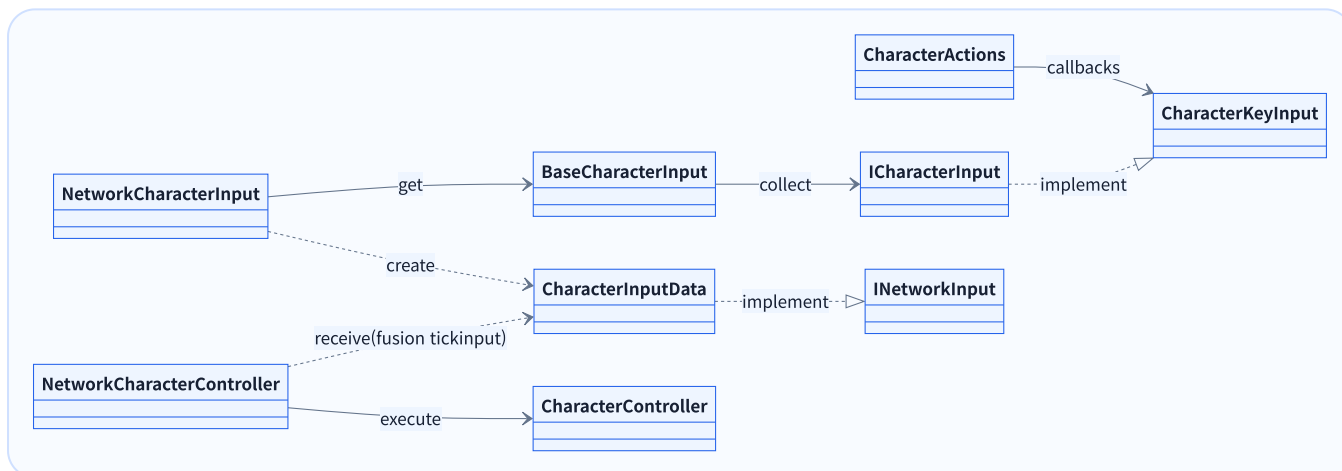
Latency

샘플 목표

- 로컬 입력 수집과 게임 로직 실행 책임 분리
- 입력 권한이 있는 클라이언트만 입력 생성
- 버튼 입력을 비트 플래그로 압축해 네트워크 전송
- 수신 입력을 이동/대시/공격 등 캐릭터 동작으로 변환
- 타임스탬프 기반 지연 측정값을 실제 스킬 실행에 전달

설계 기준

- **CharacterInputData**: 네트워크 전송용 입력 데이터
- **CharacterKeyInput** / **ICharacterInput**: Input System callback을 게임 입력 인터페이스로 변환
- **BaseCharacterInput**: 로컬 입력 소스 수집
- **NetworkCharacterInput**: 권한 검증 및 입력셋
- **NetworkCharacterController**: 권한 검증 및 네트워크 전송
- **CharacterController**: 실제 캐릭터 이동/전투 동작



프로젝트	RFist
해당 영역	HeroNetworkSystem, HeroControlSystem
샘플 범위	네트워크 입력 데이터, 입력 수집, 권한 검증, 컨트롤러 실행, 지연 측정, 동작 호출
공개 방식	시스템 흐름은 유지하되 내부 데이터, 리소스명, 일부 구현 세부사항은 제거해 채용 검토용으로 축약
제작 영상	제작영상1 제작영상2

네트워크 전송용 입력 데이터

```
public struct CharacterInputData : INetworkInput
{
    public const uint Attack = 0x0000_0001;
    public const uint AttackHold = 0x0000_0002;
    public const uint Guard = 0x0000_0004;
    public const uint AttackGuardDualInput = 0x0000_0005;
    public const uint UltimateAttack = 0x0000_0100;
    public const uint Dash = 0x0001_0000;
    public const uint FocusMode = 0x0010_0000;
    public const uint FocusChange = 0x0020_0000;

    public float Horizontal { get; set; }
    public float Vertical { get; set; }
    public float AttackTimeStamp { get; set; }
    public uint Buttons { get; set; }

    public bool IsAttack() => (Buttons & Attack) == Attack;
    public bool IsAttackHold() => (Buttons & AttackHold) == AttackHold;
    public bool IsGuard() => (Buttons & Guard) == Guard;
    public bool IsDash() => (Buttons & Dash) == Dash;
    public bool IsFocusMode() => (Buttons & FocusMode) == FocusMode;
    public bool IsFocusChange() => (Buttons & FocusChange) == FocusChange;
    public bool IsUltimateSkill() => (Buttons & UltimateAttack) == UltimateAttack;
    public bool IsAttackGuardDualInput()
        => (Buttons & AttackGuardDualInput) == AttackGuardDualInput;
}
```

포인트

- 버튼 상태를 uint bit mask로 압축
- 이동축과 버튼 입력을 하나의 구조체로 통합
- 타임스탬프를 포함해 네트워크 지연 분석 가능

실무 의도

- 매 tick 전송될 데이터 구조로 단순하게 설계.
uint 비트 마스크를 사용하여 전송량과 분기 복잡도 최소화

입력 장치 추상화

```
public interface ICharacterInput
{
    public float GetHorizontal();
    public float GetVertical();
    public bool IsAttack();
    public bool IsAttackHold();
    public bool IsGuard();
    public bool IsDash();
    ...
}

public sealed class CharacterKeyInput : MonoBehaviour, ICharacterInput
{
    private CharacterActions inputActions;
    private bool isAttack;
    private bool isAttackHold;
    ...
    private void OnEnable()
    {
        inputActions.Hero.Enable();
        inputActions.Hero.Attack.performed += OnAttackPerformed;
        inputActions.Hero.Attack.canceled += OnAttackReleased;
        ...
    }
    public float GetHorizontal() => inputActions.Hero.Move.ReadValue<Vector2>().x;
    public float GetVertical() => inputActions.Hero.Move.ReadValue<Vector2>().y;
    public bool IsAttack()
    {
        try
        {
            return isAttack;
        }
        finally
        {
            isAttack = false;
        }
    }
    public bool IsAttackHold() => isAttackHold;
    private void OnAttackPerformed(InputAction.CallbackContext ctx)
    {
        if (ctx.interaction is HoldInteraction)
            isAttackHold = true;
        else
            isAttack = true;
    }
    private void OnAttackReleased(InputAction.CallbackContext ctx)
    {
        isAttackHold = false;
    }
    ...
}
```

실무 의도

- Input System의 callback을 내부 bool cache로 저장하고, NetworkCharacterInput이 이를 소비해 CharacterInputData로 패킹
- 입력 이벤트 발생 시점과 네트워크 tick 입력 수집 시점을 분리해도 단발 입력을 안정적으로 전달할 수 있게 처리

로컬 입력 수집과 동시 입력 규칙

```

public static class BaseCharacterInput
{
    public static IReadOnlyList<ICharacterInput> CharacterInputList { get; private set; }
}

public sealed class NetworkCharacterInput : NetworkBehaviour
{
    public override void OnInput(NetworkRunner runner, NetworkInput input)
    {
        if (!Object.HasInputAuthority)
            return;

        var data = new CharacterInputData
        {
            AttackTimeStamp = runner.SimulationTime
        };

        foreach (var CharacterInput in BaseCharacterInput.CharacterInputList)
        {
            if (data.Horizontal == 0)
                data.Horizontal = CharacterInput.GetHorizontal();

            if (data.Vertical == 0)
                data.Vertical = CharacterInput.GetVertical();

            if (CharacterInput.IsDash())
                data.Buttons |= CharacterInputData.Dash;

            if (CharacterInput.IsAttack())
                data.Buttons |= CharacterInputData.Attack;

            if (CharacterInput.IsAttackHold())
                data.Buttons |= CharacterInputData.AttackHold;

            if (CharacterInput.IsGuard())
                data.Buttons |= CharacterInputData.Guard;

            if (CharacterInput.IsFocusMode())
                data.Buttons |= CharacterInputData.FocusMode;

            if (CharacterInput.IsFocusChange())
                data.Buttons |= CharacterInputData.FocusChange;

            if (CharacterInput.IsUltimateAttack())
                data.Buttons |= CharacterInputData.UltimateAttack;

            if (CharacterInput.IsAttackGuardDualInput())
                data.Buttons |= CharacterInputData.AttackGuardDualInput;
        }

        input.Set(data);
    }
}

```

실무 의도

- 공격+가드처럼 별도 액션으로 해석해야 하는 조합 입력은 수집 단계에서 별도 플래그로 변환
- BaseCharacterInput.CharacterInputList를 통해 입력 구현체를 모으면 키보드, 패드, 테스트 입력 등 입력 소스가 늘어나도 네트워크 입력 생성 흐름은 유지할 수 있게 제작

입력 권한 검증과 네트워크 전송

```
public sealed class NetworkCharacterController : NetworkBehaviour
{
    [SerializeField] private CharacterController CharacterController;
    public override void FixedUpdateNetwork()
    {
        if (GetInput<CharacterInputData>(out var input))
        {
            if (CharacterController.IsActive == false)
            {
                RPC_UpdateCharacterInputData(default);
                return;
            }
            RPC_UpdateCharacterInputData(input);
            FixedInputData = input;
        }
    }

    [Rpc(RpcSources.InputAuthority, RpcTargets.All)]
    private void RPC_UpdateCharacterInputData(CharacterInputData input)
    {
        CharacterController.Move(input.Horizontal, input.Vertical, Runner.DeltaTime);
        ProcessDash(input);
        ProcessAttack(input);
        ProcessAttackHold(input);
        ...
    }
}
```

실무 의도

- 멀티플레이 환경에서 중복입력, 원격 캐릭터 오조작 최소화를 위해 입력 권한이 없는 객체는 입력을 만들지 않음.

수신 입력을 캐릭터 명령으로 변환

```
private void ProcessAttack(CharacterInputData input)
{
    if (input.IsAttack() == false)
        return;

    var latency = Runner.SimulationTime - input.AttackTimeStamp;
    CharacterController.Attack(latency);
}

private void ProcessDash(CharacterInputData input)
{
    if (input.IsDash())
        CharacterController.Dash(input.Horizontal, input.Vertical);
}
...
```

실무 의도

- 네트워크 객체는 입력을 해석하고, 실제 이동/공격 실행은 Character 계층에서 처리
- latency처리가 필요한 동작은 실행에 넘겨 히트 판정, 보정, 로그 분석에 활용할 수 있게 처리

실제 액션 실행 계층

```
public sealed class CharacterController : MonoBehaviour
{
    [SerializeField] private CharacterMovement movement;
    [SerializeField] private CharacterAttack attack;
    ...

    private Vector2 _move;

    public void SetMove(Vector2 move)
    {
        _move = Vector2.ClampMagnitude(move, 1f);
        movement.SetMove(_move);
    }

    public void Dash(Vector2 input)
    {
        var direction = input.sqrMagnitude > 0.01f
            ? input.normalized
            : movement.Forward;

        movement.Dash(direction);
    }

    public void Attack(float latency)
    {
        attack.Execute(new AttackCommand
        {
            Direction = movement.Forward,
            NetworkLatency = latency,
        });
    }
    ...
}
```

실무 의도

- NetworkCharacterController는 입력을 수신하고, 실제 액션 실행은 CharacterController와 하위 전투/이동 컴포넌트가 담당
- 실제 프로젝트에는 캐릭터 상태 제어, 스킬 매니저, 애니메이션/이펙트/카메라 연출, 피격 및 전투 이벤트 처리 등 더 넓은 실행 계층이 구현되어 있으나, 본 코드 샘플에서는 네트워크 입력이 실제 캐릭터 동작으로 연결되는 흐름을 보여주는 데 필요한 범위만 발췌

내용 정리

- 입력 장치, 입력 수집, 네트워크 전송, 액션 실행 책임 분리
- 비트 플래그 기반으로 입력 데이터를 작게 유지
- 권한 검증으로 원격 캐릭터 입력 오염 방지
- 지연값을 게임플레이 계층까지 전달